

Numpy

NumPy is the fundamental package for scientific computing with Python. This cheat sheet is a quick reference for NumPy beginners.

Getting Started

Introduction

You'll also need to import numpy to get started:

```
import numpy as np
```

Importing/exporting

```
np.loadtxt('file.txt')
```

From a text file

```
np.genfromtxt('file.csv',delimiter=',')
```

From a CSV file

```
np.savetxt('file.txt',arr,delimiter=' ')
```

Writes to a text file

```
np.savetxt('file.csv',arr,delimiter=',')
```

Writes to a CSV file

Creating Arrays

```
np.array([1,2,3])
```

One dimensional array

<code>np.array([(1,2,3),(4,5,6)])</code>	Two dimensional array
<code>np.zeros(3)</code>	1D array of length 3 all values 0
<code>np.ones((3,4))</code>	3x4 array with all values 1
<code>np.eye(5)</code>	5x5 array of 0 with 1 on diagonal (Identity matrix)
<code>np.linspace(0,100,6)</code>	Array of 6 evenly divided values from 0 to 100
<code>np.arange(0,10,3)</code>	Array of values from 0 to less than 10 with step 3 (eg [0,3,6,9])
<code>np.full((2,3),8)</code>	2x3 array with all values 8
<code>np.random.rand(4,5)</code>	4x5 array of random floats between 0–1
<code>np.random.rand(6,7)*100</code>	6x7 array of random floats between 0–100
<code>np.random.randint(5,size=(2,3))</code>	2x3 array with random ints between 0–4

Inspecting Properties

<code>arr.size</code>	Returns number of elements in arr
<code>arr.shape</code>	Returns dimensions of arr (rows,columns)
<code>arr.dtype</code>	Returns type of elements in arr
<code>arr.astype(dtype)</code>	Convert arr elements to type dtype
<code>arr.tolist()</code>	Convert arr to a Python list
<code>np.info(np.eye)</code>	View documentation for np.eye

Copying/sorting/resaping

<code>np.copy(arr)</code>	Copies arr to new memory
---------------------------	--------------------------

<code>arr.view(dtype)</code>	Creates view of arr elements with type dtype
<code>arr.sort()</code>	Sorts arr
<code>arr.sort(axis=0)</code>	Sorts specific axis of arr
<code>two_d_arr.flatten()</code>	Flattens 2D array two_d_arr to 1D
<code>arr.T</code>	Transposes arr (rows become columns and vice versa)
<code>arr.reshape(3,4)</code>	Reshapes arr to 3 rows, 4 columns without changing data
<code>arr.resize((5,6))</code>	Changes arr shape to 5x6 and fills new values with 0

Adding/removing Elements

<code>np.append(arr, values)</code>	Appends values to end of arr
<code>np.insert(arr, 2, values)</code>	Inserts values into arr before index 2
<code>np.delete(arr, 3, axis=0)</code>	Deletes row on index 3 of arr
<code>np.delete(arr, 4, axis=1)</code>	Deletes column on index 4 of arr

Combining/splitting

<code>np.concatenate((arr1, arr2), axis=0)</code>	Adds arr2 as rows to the end of arr1
<code>np.concatenate((arr1, arr2), axis=1)</code>	Adds arr2 as columns to end of arr1
<code>np.split(arr, 3)</code>	Splits arr into 3 sub-arrays
<code>np.hsplit(arr, 5)</code>	Splits arr horizontally on the 5th index

Indexing/slicing/subsetting

<code>arr[5]</code>	Returns the element at index 5
<code>arr[2,5]</code>	Returns the 2D array element on index [2][5]

<code>arr[1]=4</code>	Assigns array element on index 1 the value 4
<code>arr[1,3]=10</code>	Assigns array element on index [1][3] the value 10
<code>arr[0:3]</code>	Returns the elements at indices 0,1,2 (On a 2D array: returns rows 0,1,2)
<code>arr[0:3,4]</code>	Returns the elements on rows 0,1,2 at column 4
<code>arr[:2]</code>	Returns the elements at indices 0,1 (On a 2D array: returns rows 0,1)
<code>arr[:,1]</code>	Returns the elements at index 1 on all rows
<code>arr<5</code>	Returns an array with boolean values
<code>(arr1<3) & (arr2>5)</code>	Returns an array with boolean values
<code>~arr</code>	Inverts a boolean array

Vector Math

<code>np.add(arr1,arr2)</code>	Elementwise add arr2 to arr1
<code>np.subtract(arr1,arr2)</code>	Elementwise subtract arr2 from arr1
<code>np.multiply(arr1,arr2)</code>	Elementwise multiply arr1 by arr2
<code>np.divide(arr1,arr2)</code>	Elementwise divide arr1 by arr2
<code>np.power(arr1,arr2)</code>	Elementwise raise arr1 raised to the power of arr2
<code>np.array_equal(arr1,arr2)</code>	Returns True if the arrays have the same elements and shape
<code>np.sqrt(arr)</code>	Square root of each element in the array
<code>np.sin(arr)</code>	Sine of each element in the array
<code>np.log(arr)</code>	Natural log of each element in the array

<code>np.abs(arr)</code>	Absolute value of each element in the array
<code>np.ceil(arr)</code>	Rounds up to the nearest int
<code>np.floor(arr)</code>	Rounds down to the nearest int
<code>np.round(arr)</code>	Rounds to the nearest int

Scalar Math

<code>np.add(arr,1)</code>	Add 1 to each array element
<code>np.subtract(arr,2)</code>	Subtract 2 from each array element
<code>np.multiply(arr,3)</code>	Multiply each array element by 3
<code>np.divide(arr,4)</code>	Divide each array element by 4 (returns np.nan for division by zero)
<code>np.power(arr,5)</code>	Raise each array element to the 5th power

Statistics

<code>np.mean(arr,axis=0)</code>	Returns mean along specific axis
<code>arr.sum()</code>	Returns sum of arr
<code>arr.min()</code>	Returns minimum value of arr
<code>arr.max(axis=0)</code>	Returns maximum value of specific axis
<code>np.var(arr)</code>	Returns the variance of array
<code>np.std(arr,axis=1)</code>	Returns the standard deviation of specific axis
<code>arr.corrcoef()</code>	Returns correlation coefficient of array

Top Cheatsheet

Python Cheatsheet

Quick Reference

Vim Cheatsheet

Quick Reference

JavaScript Cheatsheet

Quick Reference

Bash Cheatsheet

Quick Reference

Recent Cheatsheet

InfrontJS Cheatsheet

Quick Reference

Remote Work Revolution Cheatsheet

Quick Reference

Homebrew Cheatsheet

Quick Reference

PyTorch Cheatsheet

Quick Reference